

Alexandre Plana

@alexandre-plana

Développeur full-stack, expertise front-end

Applications industrielles · UI/UX · Multiplateforme

À propos

Développeur spécialisé dans les applications industrielles, j'ai construit une expertise front-end et UI/UX que j'étends aujourd'hui au développement full-stack. Sur Xistar, j'interviens de l'interface opérateur aux services qui intègrent et diffusent les données opérationnelles. J'aborde le développement en reliant usages métier, architecture et contraintes techniques, avec une attention portée à la cohérence du produit, de la maquette au code livré.

Consultant R&D · Datacorp / X-PERT

France

alexandre.plana.pro@gmail.com

+33 6 71 03 97 84

Langues

Français Natif · Anglais Technique

Compétences clés

LANGAGES

typescript · go · c# · python

FRONT-END & VISUALISATION

react · vue 3 · single-spa · dockview · maplibre · deck.gl · unity 3d

SERVICES & INTEROPÉRABILITÉ

api rest · mqtt · ais · nmea · cot · mavlink

QUALITÉ & OUTILS

jest · vitest · sonarqube · gitlab ci · docker · git · postman · jira · figma

Centres d'intérêt

🎮 jeux · 🚵 vtt · 🥾 rando · 🍳 cuisine

CV en ligne
alexandre-plana.github.io/resume/

📌 Parcours

Datacorp — 2026 — Consultant R&D – X-PERT

x-pert/xistar — Développeur full-stack

Xistar – Marine Command Center · 2026

web · multi-écran · temps réel · react · typescript · go · docker · gitlab ci

x-pert/ai-workflow — Conception & développement IA

Workflow d'exécution d'agents IA · 2026

ia · bmad · docker · gitlab ci · react

x-pert/xpert-design-system — Conception UI & architecture front-end

Design system de X-PERT – de Figma aux applications · 2026

web · ux/ui · figma · react · shadcn · npm

Datacorp — 2017 – 2026 — Consultant R&D – Schneider Electric

schneider-electric/iiot-edge-ieca — Développeur front-end

Industrial Automation Edge (IECA) · 2023 – 2026

web · vue 3 · typescript · single-spa · pinia · jest · sonarqube

schneider-electric/aoa-ar-app — Développeur C# / Unity

Augmented Operator Advisor (AOA) – Application AR industrielle · 2017 – 2023

3d / xr · unity · c# · ar · mobile · offline

Oreka Ingénierie — 2013 – 2017 — Développeur C# / Unity 3D

suez/radioprotection — Développeur C# / Simulation de formation

Formation en Radioprotection – Nuclear Safety Training · 2014 – 2015

serious game · radioprotection · nucléaire · unity · c#

simulhom — Conception fonctionnelle & UX/UI

Outil de prévention des TMS (Troubles Musculo-Squelettiques) · 2015 – 2017

ux/ui · ergonomie · simulation 3d · unity · c#

chu-caen/emsim-basic — Développeur C# / Simulation médicale 3D

Radioprotection pour la Chirurgie Interventionnelle · 2013 – 2015

simulation 3d · radioprotection · médical · unity · c#

areva/emem — Développeur C# / Simulation 3D

EMEM (Enceinte Mobile d'Évacuation de Matériel) – Nuclear Safety · 2014 – 2016

simulation 3d · nucléaire · unity · c# · wpf

edf/aeroball — Développeur C# / Serious game

Aeroball – Serious Game pour poste de contrôle nucléaire · 2014 – 2015

serious game · nucléaire · unity · c#

suez/maintenance-generateurs-de-vapeurs — Développeur C# / Serious game

Maintenance Générateurs de Vapeurs – Serious Game pour arrêts de tranche · 2014 – 2015

serious game · nucléaire · unity · c#

📌 Vue.js · micro-frontend – Schneider Electric

Datacorp 2017 – 2026
Consultant R&D – Schneider Electric

📁 **schneider-electric/iiot-edge-ieca** — Développeur front-end

Industrial Automation Edge (IECA) · 2023 – 2026

Au sein d'une équipe `#scrum` pluridisciplinaire, j'ai développé des applications Edge organisées en `#microfrontend` avec `#vue3`, `#typescript` et `#single-spa`. Mon périmètre couvrait principalement les interfaces et leur architecture, avec quelques extensions de fonctionnalités back-end. J'intervenais à la fois sur les domaines fonctionnels de la plateforme, les composants partagés, les tests et le suivi de la qualité du code.

ACTIONS CLÉS

- Participer au développement des fonctionnalités liées aux connecteurs, aux calculs, aux conditions, aux alertes et au scripting.
- Concevoir en `#vue3` et `#typescript` des composants partagés, dont une table configurable, un socle de routage et un sélecteur de propriétés.
- Étendre ponctuellement les fonctionnalités back-end nécessaires aux évolutions prises en charge côté front-end.
- Développer avec `#jest` les tests unitaires des composants, des stores `#pinia` et des fonctions utilitaires, puis suivre les performances et la qualité du code avec `#sonarqube`.
- Concevoir, lors d'un IP Sprint, un prototype d'application `#mobile` destiné à transmettre les alertes aux opérateurs d'astreinte en dehors de la plateforme IECA.

`vue3` `ts` `single-spa` `docker` `jest` `html` `css` `git` `figma` `jira`

MICRO-FRONTENDS IECA SUR LESQUELS J'AI TRAVAILLÉ

J'ai travaillé sur chacun des micro-frontends ci-dessous, tous développés en `#vue3` et `#typescript` et chargés comme applications `#single-spa`, avec `#pinia`, Vue Router et le design system Quartz de Schneider Electric. Les applications récentes partagent la même architecture : composants container/présentation, mappers DTO → modèle typés, un store par domaine et des formulaires validés avec vee-validate et yup ou zod.

SOCLE PARTAGÉ	
🧩 `@ieca/ieca-web-utils` **Librairie partagée**	Composants MainTable, PropertiesPicker, Stepper, RoutableHeader avec fil d'Ariane, dialogues d'import et de confirmation, zone de dépôt ; stores communs (dialogues, notifications, droits, registre d'applications) ; client HTTP avec intercepteurs ; paquets de typages et d'utilitaires de test. `vue3` `typescript` `tailwind` `axios` `npm`
🧩 `ieca-vue3-ui-template` **Gabarit de micro-frontend**	Point de départ des nouvelles applications : routeur filtré selon les droits d'écriture, stores de dialogues, notifications et droits, configuration `#single-spa` et `#jest`. `vue3` `single-spa` `jest`
DONNÉES & CONNECTIVITÉ	
🧩 `ieca-connectivity-ui` **Connectivité OT/IT**	La plus grosse application (96 composants) : connecteurs OPC UA et Modbus créés, dupliqués et configurés par formulaires générés, variables éditées dans une grille ag-grid aux cellules dynamiques, import CSV, actions, et parcours interfaces → groupes → propriétés côté IT. `vue3` `ag-grid` `monaco` `yup` `tailwind`
🧩 `ieca-data-management-ui` **Gestion des données**	Arbre du modèle de données filtrable (filtre conservé en session), propriétés et leurs affectations (variable, calcul, source), ajout de propriétés et de niveaux, import, historique exportable et écriture de valeurs. `vue3` `monaco` `yup` `tailwind`
🧩 `ieca-datamodel-ui` **Modèle de données**	Éditeur arborescent du modèle de données : création et suppression de niveaux, association de propriétés via PropertiesPicker, import. Réécriture de l'ancienne application (actifs et équipements) vers la nouvelle API. `vue3` `pinia` `tailwind`

TRAITEMENTS & SUPERVISION	
🧩 `ieca-computation-ui` **Calculs**	Variables calculées lancées ou arrêtées depuis la liste ; assistant en étapes : propriétés, formule validée côté serveur avec catalogue de fonctions, puis fenêtre temporelle glissante ou fixe, avec aperçus. `vue3` `typescript` `pinia`
🧩 `ieca-condition-ui` **Conditions**	Règles Si/Alors construites en étapes : propriétés et niveau de qualité, formule validée côté serveur, puis action (application, commande ou script) ; transformation de JSON Schema pour les paramètres. `vue3` `typescript` `json schema`
🧩 `ieca-scripting-ui` **Scripting**	Scripts créés par formulaire généré et édités dans Monaco avec suggestions, activés, lancés ou annulés ; exécutions et journaux analysés ; gardes de navigation pour les droits et l'édition. `vue3` `monaco` `cron` `yup`
🧩 `ieca-alert-manager-ui` **Alertes**	Liste des alertes et historique d'activité, acquittement et clôture, suppression groupée par JSON Patch et export CSV. `vue3` `typescript` `floating-vue`
ADMINISTRATION DE LA PLATEFORME	
🧩 `ieca-platform-configuration-ui` **Configuration de la plateforme**	Certificat et clé du serveur, occupation disque de la base, export et import de la configuration avec rapport d'erreurs, paramètres des applications par formulaires générés. `vue3` `monaco` `yup` `tailwind`
🧩 `ieca-mail-ui` **Messagerie SMTP**	Configuration du serveur SMTP (hôte, sécurité, identifiants, certificat vérifié avec node-forge) et envoi d'un message de test avec état de connexion. `vue3` `typescript` `node-forge`
🧩 `ieca-overview-ui` **Vue d'ensemble**	Page « Applications » : cartes des applications Edge installées avec leur éditeur et leur licence. `vue3` `tailwind`
PROTOTYPES	
🧩 `alexandre-plana/ip-sprint-*` **IP Sprint – données sur mobile (2024)**	Version mobile de la gestion des données d'IECA, avec Capacitor pour `#android` et iOS puis Quasar, et expérimentation d'un shell mobile capable de charger des micro-frontends `#single-spa`. `vue3` `capacitor` `quasar` `mobile`
🧩 `ip-sprint · notifications` **IP Sprint – notifications d'astreinte**	Prototype qui fait sortir les alertes d'IECA de la plateforme : elles alimentent un flux dédié au mobile et deviennent des notifications push, reçues sur une application mobile conçue pour l'occasion, par exemple par un opérateur d'astreinte. `mobile` `notifications push`

📌 Développement web – X-PERT

Datacorp 2026
Consultant R&D – X-PERT

📁 x-pert/xistar

Développeur full-stack

Xistar – Marine Command Center · 2026

Xistar réunit la situation tactique, la conduite de missions et de drones, la gestion des alertes, le pilotage multi-écran et le rejeu des opérations. Je participe à la conception fonctionnelle à partir des besoins opérationnels, en définissant les parcours opérateur, les interactions et le comportement attendu des fonctionnalités. L'interface repose sur `#typescript`, `#react` et `#dockview` ; la cartographie utilise `#maplibre` et `#deckgl` pour afficher une symbologie OTAN `#app-6d`, une grille `#mgrs` et la bathymétrie. Les services `#go` ingèrent des flux partenaires `#ais`, `#nmea`, `#cot` et `#mavlink`, transmettent des ordres et publient des informations via `#mqtt` et `#api-rest`. Les profils de déploiement et la communication entre instances permettent d'adapter le produit aux contextes défense, portuaire et démonstration.

Multi-écran & temps réel

Application multi-écran pilotée depuis un poste maître : ensembles d'écrans, tableau de mission et envoi de vues à distance, avec une gestion centralisée et en temps réel de ce que chaque écran affiche. Les situations, alertes et positions arrivent en continu par `#mqtt` et `#socketio`, et le rejeu temporel permet de revoir une opération pour l'analyse et les débriefings.

Conteneurs & pipelines

Interface et services `#go` conteneurisés avec `#docker`, workflow de développement et pipelines `#gitlab-ci` pour construire et livrer les images.

🗂️ x-pert/xistar-fleet-sim

Simulateur de drones & flotte navale · 2026

Développement d'un simulateur de drones et de flotte navale destiné à alimenter Xistar avec des flux représentatifs des conditions d'exploitation. Réalisé en `#typescript` sous `#nodejs`, il génère positions, états et détections, puis les publie via `#mqtt` pour les démonstrations et la mise au point des fonctionnalités, sans recourir à des mocks statiques.

🗂️ x-pert/xistar-mobile

App mobile – alertes & ordres · 2026

Développement d'une application mobile permettant aux opérateurs de recevoir les alertes BAM/CAM et de transmettre des ordres depuis le terrain. Réalisée en `#typescript` avec `#react`, elle prend en charge l'escalade des alertes et l'émission d'ordres de retour, et communique avec Xistar via `#socketio`.

🗂️ x-pert/xistar-wall-emulator

Émulateur de mur d'écrans · 2026

Développement d'un émulateur de mur d'écrans permettant de concevoir et démontrer les usages multi-vues de Xistar sans disposer d'une salle équipée. Réalisé en `#typescript` avec `#react` et `#vite`, il reproduit des ensembles d'écrans, un tableau de mission et l'envoi distant de vues depuis le poste maître.

ACTIONS CLÉS

- Participer à la conception fonctionnelle de Xistar à partir des besoins opérationnels : définir les parcours opérateur, les interactions et le comportement attendu des fonctionnalités.
- Concevoir en `#typescript` et `#react` l'espace de travail opérateur sous forme de panneaux `#dockview`, puis intégrer la cartographie `#maplibre` `#deckgl` en symbologie OTAN `#app-6d`.
- Développer le système d'alertes et ses mécanismes d'escalade, avec une `#api` dédiée et l'envoi de notifications par e-mail.
- Assurer dans les services `#go` l'interopérabilité avec les systèmes partenaires : ingestion des flux `#ais`, `#nmea`, `#cot` et `#mavlink`, transmission des ordres et publication via `#mqtt` et `#api-rest`.
- Développer l'application multi-écran et sa gestion centralisée depuis un poste maître, puis mettre en place le rejeu temporel destiné à l'analyse, aux rapports de situation et aux débriefings.

`typescript` `react` `go` `dockview` `maplibre` `deck.gl` `mqtt` `api rest` `docker` `gitlab ci`

📁 x-pert/xpert-design-system

Conception UI & architecture front-end

Design system de X-PERT – de Figma aux applications · 2026

Le design system de X-PERT couvre les applications techniques, le site vitrine et l'habillage des documents officiels et des vidéos. Les composants sont conçus sous `#figma`, développés en `#typescript` avec `#react` et `#shadcn`, puis publiés sur un registre `#npm` privé. La bibliothèque réunit les éléments courants – cartes, boutons, interrupteurs et tables – et les composants propres aux interfaces techniques : graphiques et treemaps, flux d'activité, frises chronologiques, cartographie, onglets et panneaux redimensionnables. La documentation interne décrit leur intégration dans les applications et leur utilisation dans les documents officiels.

ACTIONS CLÉS

- Concevoir sous `#figma` les interfaces et composants du design system pour les applications techniques et le site vitrine.
- Développer la bibliothèque en `#typescript` et `#react` à partir de `#shadcn` : cartes, boutons, interrupteurs, tables, graphiques, treemaps, flux d'activité, frises chronologiques, cartographie, onglets et panneaux redimensionnables.
- Publier les composants sur un registre `#npm` privé pour les distribuer dans les applications de la société.
- Définir l'habillage des documents officiels et des vidéos, puis rédiger la documentation interne destinée aux applications et aux documents.

`typescript` `react` `shadcn` `figma` `npm` `design system`

📁 x-pert/ai-workflow

Conception & développement IA

Workflow d'exécution d'agents IA · 2026

J'ai conçu un workflow de développement assisté par agents IA en prenant la méthode `#bmad` comme base. Mon projet personnel lasit m'a par ailleurs permis de me familiariser avec le pilotage d'agents, le suivi des exécutions et les boucles d'amélioration ; cette expérience a nourri la conception du projet. Chaque projet dispose de sa propre image `#docker` pour exécuter les agents dans un environnement isolé ; les runs sont distribués vers une machine dédiée via `#gitlab-ci`. Un outil en `#react` et `#typescript` exploite les hooks des agents pour suivre leur progression au fil de l'exécution. Les rapports de session alimentent une boucle d'auto-amélioration des agents et du workflow.

ACTIONS CLÉS

- Structurer un workflow de développement assisté par agents IA à partir des principes de `#bmad`.
- Isoler l'exécution des agents dans des conteneurs `#docker` reposant sur une image propre à chaque projet.
- Configurer avec `#gitlab-ci` le déclenchement et la distribution des runs vers une machine dédiée.
- Développer en `#react` et `#typescript` un outil de suivi alimenté par les hooks des agents pour restituer leur progression au fil de l'exécution.
- Analyser les rapports de session et réinjecter automatiquement les enseignements identifiés afin d'améliorer les agents et le workflow.

`bmad` `docker` `gitlab ci` `react` `typescript` `ai agents`

📌 Application AR industrielle – Schneider Electric

Datacorp 2017 – 2026

Consultant R&D – Schneider Electric

📁 schneider-electric/aoa-ar-app

Développeur C# / Unity

Augmented Operator Advisor (AOA) – Application AR industrielle · 2017 – 2023

Pendant six ans, j'ai développé en `#csharp` et `#unity` les interfaces et l'architecture front-end d'une application `#ar` destinée aux opérateurs industriels. Le périmètre couvrait les versions `#android`, `#ios` et `#windows`, leur personnalisation pour les clients `#oem`, le fonctionnement `#offline` et la préparation des applications pour les différentes plateformes de distribution. J'ai également contribué au serveur de projets en `#python` et à la communication `#http` avec l'application.

ACTIONS CLÉS

- Concevoir en `#csharp` et `#unity` l'architecture front-end et une interface adaptée à `#android`, `#ios` et `#windows`, en modes portrait et paysage.
- Développer le fonctionnement `#offline` et la synchronisation `#http`, puis contribuer aux évolutions du serveur de projets en `#python` et à la gestion des erreurs réseau.
- Assurer la rétrocompatibilité des projets par la lecture des anciens formats et leur migration à la volée.
- Mettre en place un système de styles externalisé pour produire des versions personnalisées aux couleurs des clients `#oem`.
- Créer sous `#unity` un outil de tests unitaires intégré aux étapes précédant et suivant les builds.

`unity` `c#` `ar` `mobile` `uiux` `figma` `agile` `jira`

📌 Serious games & simulation – Oreka Ingénierie

Oreka Ingénierie `2013 - 2017`

Développeur C# / Unity 3D

📁 oreka/xr-projects

Développeur C# / Unity 3D

Projets XR industriels – EDF · RTE · AREVA · DCNS · SUEZ · 2013 – 2017

Pendant quatre ans, j'ai participé à la conception fonctionnelle et au développement d'applications interactives pour l'industrie, le nucléaire et la défense. À partir des besoins clients et des procédures métier, je contribuais à définir les parcours, les scénarios, les interactions et les maquettes fonctionnelles des serious games `#seriousgame` et des simulateurs. J'assurais ensuite leur réalisation en `#csharp` sous `#unity`, l'intégration des modèles issus de `#solidworks` et `#3dsmax`, l'optimisation CPU/GPU et la préparation des livraisons.

ACTIONS CLÉS

- Participer à la conception fonctionnelle : analyser les besoins clients, étudier leur faisabilité technique et formaliser les parcours, les interactions et les maquettes.
- Concevoir des serious games `#seriousgame` et des simulateurs à partir des procédures métier, puis structurer les scénarios à l'aide de graphes d'états, de conditions et de déclencheurs.
- Développer en `#csharp` sous `#unity` les interfaces et les systèmes interactifs 2D/3D, dont les interactions propres aux dispositifs `#vr` et `#ar`.
- Intégrer et optimiser des modèles issus de la `#cao` avec `#solidworks` et `#3dsmax` : maillages, niveaux de détail, lightmaps, matériaux et shaders.
- Optimiser les performances CPU/GPU avec les outils de profiling de `#unity`, puis préparer les builds, corriger les anomalies et accompagner les démonstrations client.

`unity` `c#` `wpf` `winform` `solidworks` `3dsmax` `vr` `uiux` `figma` `desktop`

PROJETS PHARES

📁 oreka/dosicase

Conception UX/UI

Dosicase – Simulateur de radioactivité pour chantiers école · 2015 – 2017

Dosicase met la simulation au service de la formation pratique en radioprotection. J'ai conçu l'expérience et l'interface `#uiux` pour présenter les situations et les risques radiologiques de manière pédagogique, puis contribué à leur intégration dans le simulateur en `#csharp` sous `#unity`. Le travail portait autant sur l'organisation des informations que sur les interactions nécessaires à l'apprentissage en chantier-école.

ACTIONS CLÉS

- Structurer le parcours pédagogique à partir des objectifs de formation en radioprotection.
- Concevoir l'interface `#uiux` et organiser les informations liées aux situations et aux risques radiologiques.
- Définir les interactions du simulateur nécessaires à l'apprentissage pratique en chantier-école.
- Participer à l'intégration de l'expérience dans l'application en `#csharp` sous `#unity`.

`uiux` `design` `web` `radioprotection` `dosimetry` `figma`

📁 suez/radioprotection

Développeur C# / Simulation de formation

Formation en Radioprotection – Nuclear Safety Training · 2014 – 2015

Le module proposait des scénarios d'accident simulés pour former aux procédures de radioprotection et de sécurité nucléaire. Mon travail portait sur la structuration des séquences de formation, la modélisation du déroulement des scénarios et la gestion des équipes de secours. J'ai ensuite développé et intégré ces mécanismes en `#csharp` sous `#unity` dans un environnement 3D.

ACTIONS CLÉS

- Traduire les procédures de sécurité nucléaire en séquences de formation interactives.
- Concevoir les scénarios d'accident simulés et leur déroulement.
- Implémenter la gestion des équipes de secours au sein de la simulation.
- Développer en `#csharp` sous `#unity` le module de formation et son environnement 3D.

`unity` `c#` `3d-temps-réel` `training` `education`

📁 simulhom

Conception fonctionnelle & UX/UI

Outil de prévention des TMS (Troubles Musculo-Squelettiques) · 2015 – 2017

Simulhom est un simulateur ergonomique 3D conçu pour analyser les postures de travail et visualiser les risques de TMS. J'ai participé à la conception fonctionnelle du produit avec des ergonomes, puis conçu les parcours et l'interface `#uiux` afin de rendre les indicateurs posturaux lisibles et exploitables. J'ai également contribué au développement de l'application en `#csharp` sous `#unity` et à l'intégration des retours visuels dans la scène 3D.

ACTIONS CLÉS

- Participer à la conception fonctionnelle du produit avec des ergonomes, en formalisant les usages et les scénarios d'analyse.
- Concevoir les parcours, l'interface `#uiux` et les retours visuels associés aux risques posturaux.
- Développer en `#csharp` sous `#unity` les interactions et les composants du simulateur 3D.
- Intégrer l'analyse posturale et restituer les indicateurs de risque au fil de la simulation.

`unity` `c#` `3d-temps-réel` `vr` `ergonomics`

AUTRES RÉALISATIONS

🗂️ chu-caen/emsim-basic

Radioprotection pour la Chirurgie Interventionnelle · 2013 – 2015

Développement d'une simulation médicale 3D destinée à former les équipes de chirurgie interventionnelle aux pratiques de radioprotection. J'ai contribué à la représentation de la dosimétrie au cours des scénarios, à la visualisation des zones exposées aux rayonnements et à l'intégration des protocoles opérateur en `#csharp` sous `#unity`.

🗂️ areva/emem

EMEM (Enceinte Mobile d'Évacuation de Matériel) – Nuclear Safety · 2014 – 2016

Développement d'une simulation 3D interactive de l'EMEM, une enceinte mobile destinée à l'évacuation de matériel en environnement nucléaire. J'ai contribué à la représentation des flux et des matériaux radioactifs ainsi qu'à la conception des interfaces de contrôle pour les opérateurs, en `#csharp` avec `#unity` et `#wpf`.

🗂️ edf/aeroball

Aeroball – Serious Game pour poste de contrôle nucléaire · 2014 – 2015

Développement d'un serious game `#seriousgame` destiné à former les opérateurs à l'utilisation du poste de contrôle Aeroball en environnement nucléaire. J'ai participé à la traduction des procédures opérationnelles en scénarios interactifs, puis à leur réalisation sous `#unity` en `#csharp` et à la modélisation 3D de l'environnement de formation.

🗂️ suez/maintenance-generateurs-de-vapeurs

Maintenance Générateurs de Vapeurs – Serious Game pour arrêts de tranche · 2014 – 2015

Développement d'un serious game `#seriousgame` destiné à préparer les interventions de maintenance des générateurs de vapeur pendant les arrêts de tranche. J'ai participé à la transposition des procédures de maintenance nucléaire en scénarios interactifs, puis à leur réalisation en `#csharp` sous `#unity` dans un environnement 3D.

🛠️ Projets persos

🛠️ PILOTAGE DU DÉVELOPPEMENT ASSISTÉ PAR IA — en service

iasit

Iasit est un outil de pilotage du développement assisté par des agents IA. Je l'ai conçu pour structurer le cycle préparation, exécution, vérification, correction et acceptation, suivre les bugs sous forme de findings, puis capitaliser sur les problèmes récurrents en les promouvant en learned rules. Ces règles sont réinjectées dans les futurs prompts afin d'améliorer progressivement les exécutions. L'application repose sur `#react` `#typescript` et un runtime `#nodejs`.

🗓 2026 · Conception produit & développement full-stack

`react` `typescript` `nodejs` `zod` `sigma.js` `api rest` `mcp`

🛠️ SANDBOX PÉDAGOGIQUE 3D actif

vorrery

Vorrery est un sandbox pédagogique 3D conçu pour expérimenter, observer et comprendre par la manipulation. L'utilisateur construit ses propres systèmes, ajuste leurs paramètres et visualise immédiatement les conséquences de ses choix. Le produit combine exploration libre, expériences guidées et parcours adaptés aux enfants dans une interface `#react` `#typescript` rendue avec `#threejs` et `#react-three-fiber`.

🗓 2026 · Conception produit & développement front-end 3D

react · typescript · three.js · r3f · web workers

🛠️ APPLICATION AUTO-HÉBERGÉE en conception

kin

Conception d'une mémoire numérique familiale privée et auto-hébergée, destinée à conserver photos, vidéos et documents sur un serveur domestique. Le produit prévoit une galerie Android, une administration locale et un accès mobile distant, avec une attention particulière portée à la fiabilité du stockage et à la maîtrise des données.

🗓 2026 · Conception produit & architecture

node · typescript · android · api rest · sqlite

🎓 Formations

📚 FORMATION

React · TypeScript

Datacorp · 2026

Formation approfondie à React et TypeScript, centrée sur les pratiques modernes du développement front-end

📚 FORMATION

TypeScript · Docker · Vue.js

Datacorp · 2023

Formations professionnelles à TypeScript, Docker et Vue.js, appliquées au développement d'applications web

📚 STAGE

Polymorph

Stage de fin de licence · 2013

Design graphique · Communication visuelle

🎓 DIPLÔME

Licence 3D temps réel

3DI Laval · 2012–2013

Unity 3D · Réalité virtuelle · 3D temps réel

🎓 DIPLÔME

BTS Communication visuelle – option Multimédia

3DI Laval · 2010–2012

Multimédia · Web · Vidéo